

Perfect. Here are the **first 20 Senior Front-End interview questions with interview-ready answers.**

1. What is React?

Answer

React is a JavaScript library for building user interfaces. It uses a component-based architecture where the UI is broken into reusable components. React updates the UI efficiently using the Virtual DOM and reconciliation.

Key Points

- Component-based
 - Declarative
 - Virtual DOM
 - One-way data flow
 - Reusable components
-

2. How does React rendering work?

Answer

When a component's state or props change:

1. React calls the component function again.
2. A new Virtual DOM tree is created.
3. React compares it with the previous Virtual DOM (Diffing).
4. Only the changed DOM elements are updated in the real DOM.

This process is called **Reconciliation**.

3. What causes a React component to re-render?

A component re-renders when:

- State changes
- Props change
- Parent component re-renders
- Context value changes
- A forced update occurs

React **does not** re-render just because a variable changes.

JavaScript

```
let count = 0;

count++; // Doesn't trigger render

setCount(count + 1); // Triggers render
```

4. What is the Virtual DOM?

The Virtual DOM is an in-memory representation of the real DOM.

Instead of updating the browser DOM immediately, React:

- Creates a new Virtual DOM
- Compares it with the previous one
- Updates only the changed elements

Benefits:

- Faster updates
- Fewer DOM operations
- Better performance

5. What is Reconciliation?

Reconciliation is React's algorithm for updating the UI efficiently.

Process:

State Changes

↓

Render

↓

New Virtual DOM

↓

Compare with Old Virtual DOM

↓

Update only changed nodes

6. How does React Diffing work?

React compares two Virtual DOM trees.

Rules:

- Different element types → Replace entire subtree.
- Same element type → Update only changed attributes.
- Children are compared recursively.
- Keys help React identify moved items.

Example:

JavaScript

Old

```
<li>A</li>  
<li>B</li>
```

New

```
<li>A</li>  
<li>C</li>
```

Only the second `<li>` changes.

7. Why are keys important?

Keys uniquely identify list items.

Without keys:

React may recreate every item.

With keys:

React can identify:

- Added items

- Removed items
- Reordered items

Good:

JavaScript

```
users.map(user =>  
  <User key={user.id} />  
)
```

Bad:

JavaScript

```
key={index}
```

Index keys can cause bugs if the list order changes.

8. What is React Fiber?

Fiber is React's rendering engine introduced in React 16.

Benefits:

- Interruptible rendering
- Prioritized updates
- Better animations
- Concurrent rendering
- Improved scheduling

Instead of blocking the browser, Fiber can pause rendering and resume later.

9. Explain useState.

`useState` adds state to functional components.

JavaScript

```
const [count, setCount] = useState(0);
```

- `count` is the current state.
- `setCount` updates the state.
- Calling `setCount` schedules a re-render.

Never mutate state directly.

Wrong:

JavaScript

```
count++;
```

Correct:

JavaScript

```
setCount(count + 1);
```

10. Why is state immutable?

React detects changes by comparing object references.

Wrong:

JavaScript

```
user.name = "John";  
setUser(user);
```

Reference remains the same.

Correct:

JavaScript

```
setUser({  
  ...user,  
  name: "John"  
});
```

New object → React detects the change.

11. What is useEffect?

`useEffect` performs side effects after rendering.

Examples:

- Fetch API
- Timers
- Event listeners
- WebSocket connections
- Local storage

Example:

JavaScript

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

Runs after the first render.

12. When does useEffect run?

No dependency array:

JavaScript

```
useEffect(() => {})
```

Runs after every render.

Empty dependency:

JavaScript

```
useEffect(() => {}, []);
```

Runs only once after the initial render.

With dependencies:

JavaScript

```
useEffect(() => {}, [user]);
```

Runs whenever `user` changes.

Cleanup:

JavaScript

```
useEffect(() => {  
  window.addEventListener(...);  
  
  return () => {  
    window.removeEventListener(...);  
  };  
}, []);
```

Cleanup runs before the effect re-runs and when the component unmounts.

13. Difference between `useEffect` and `useLayoutEffect`?

`useEffect`

Runs after the browser paints the UI.

Render

↓

Paint

↓

`useEffect`

`useLayoutEffect`

Runs synchronously after DOM updates but **before** the browser paints.

Render

↓

DOM Update

↓

`useLayoutEffect`

↓

Paint

Use it for DOM measurements or preventing visual flicker.

14. What is useMemo?

`useMemo` memoizes the **result** of an expensive calculation.

JavaScript

```
const total = useMemo(() => {  
  return calculateTotal(items);  
}, [items]);
```

Use it when computations are expensive and dependencies change infrequently.

15. What is useCallback?

`useCallback` memoizes a **function**.

JavaScript

```
const handleClick = useCallback(() => {  
  save();  
}, []);
```

Useful when passing callbacks to memoized child components to avoid unnecessary re-renders.

16. Difference between useMemo and useCallback?

useMemo

useCallback

Memoizes a value

Memoizes a function

Returns computed result

Returns function reference

Used for expensive calculations

Used to prevent unnecessary callback recreation

17. What is React.memo?

`React.memo` prevents a component from re-rendering if its props haven't changed.

JavaScript

```
const User = React.memo(({ name }) => {  
  return <h1>{name}</h1>;  
});
```

It's useful for components that render frequently with unchanged props.

18. What are controlled and uncontrolled components?

Controlled Component

React controls the input value.

JavaScript

```
const [name, setName] = useState("");  
  
<input  
  value={name}  
  onChange={e => setName(e.target.value)}  
>
```

Uncontrolled Component

The DOM manages the input value.

JavaScript

```
const inputRef = useRef();  
  
<input ref={inputRef} />
```

Controlled components are generally preferred because they make form state easier to validate and manage.

19. What is lifting state up?

Lifting state up means moving shared state to the nearest common parent so multiple child components can access and update it.

Parent

└─ Search

└─ ProductList

Instead of each child having its own state, the parent owns the state and passes it down via props.

Benefits:

- Single source of truth
- Easier synchronization
- Better data flow

20. What are custom hooks?

A custom hook is a reusable function that encapsulates stateful logic using React hooks.

Example:

JavaScript

```
function useWindowWidth() {  
  const [width, setWidth] = useState(window.innerWidth);  
  
  useEffect(() => {  
    const onResize = () => setWidth(window.innerWidth);  
  
    window.addEventListener("resize", onResize);  
  
    return () => window.removeEventListener("resize", onResize);  
  }, []);  
  
  return width;  
}
```

Usage:

JavaScript

```
const width = useWindowWidth();
```

Custom hooks let you reuse logic across components without duplicating code.

Senior-Level Tips

Interviewers often ask follow-up questions after these basics. Be prepared to explain:

- Why React uses the Virtual DOM instead of updating the DOM directly.
- When **not** to use `useMemo` or `useCallback` (they have overhead and shouldn't be used everywhere).
- Why immutability matters for React's change detection.
- How to optimize a slow React application using `React.memo`, code splitting, lazy loading, virtualization, and proper state management.
- Common causes of unnecessary re-renders and how to identify them with React DevTools.

These 20 questions cover many of the React fundamentals expected from a Senior Front-End Developer. The next section will move into **JavaScript fundamentals (closures, event loop, promises, async/await, `this`, prototypes, and more)**, which are also heavily tested.